

Développement Mobile

(Android & React Native)

Said Charfi

Maitre de conférences habilité
Département Informatique, Faculté des Sciences,
Université Ibnou Zohr
Agadir, Maroc

Filiere : ILTI5

s.charfi@uiz.ac.ma

Introduction générale

Le développement mobile connaît une évolution rapide, portée par l'amélioration du matériel, l'essor des frameworks multiplateformes, et la démocratisation des smartphones. Ce cours enrichi inclut désormais :

- des schémas explicatifs,
- des illustrations,
- des encadrés pédagogiques,
- des exemples de code avancés,
- des compléments sur les bonnes pratiques et architectures modernes.

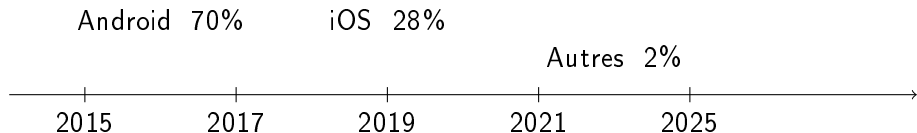
L'objectif principal est d'offrir une vision claire, illustrée et moderne du développement Android et cross-platform.

Parts de marché

- Android : $\approx 72\%$
- iOS : $\approx 27\%$
- Autres : $\approx 1\%$

Tendances actuelles :

- Flutter et React Native gagnent en popularité.
- Kotlin Multiplatform permet le partage de code avec performance native.



évolution historique

- 2000s : Symbian, BlackBerry OS
- 2007 : Lancement de l'iPhone → iOS
- 2008 : Android 1.0
- 2010–2012 : Windows Phone (disparu)
- Aujourd'hui : Android $\approx 72\%$, iOS $\approx 27\%$

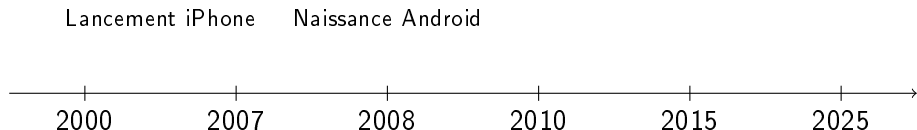


Figure – Frise chronologique des OS mobiles

- Accessibilité et performance
- Expérience utilisateur (UX)
- Compatibilité multiplateforme

Les 4 approches du développement mobile

Il existe 4 manières d'aborder le développement mobile selon les plateformes :

- Développement natif
- Développement web
- Développement hybride
- Développement multi-plateforme natif

Le développement natif utilise les langages, API et outils spécifiques à chaque plateforme.

Avantages :

- Application performante et de grande qualité
- Accès total aux spécificités du terminal mobile
- Interface homogène avec le système
- Applications très réactives
- Accès complet aux APIs et ressources

Inconvénients :

- Coûts élevés (développement + maintenance multipliés)
- Délais plus importants
- Nécessité de compétences spécifiques sur chaque plateforme

Avantages :

- Accessible depuis toutes les plateformes
- Réduction du coût et du temps de développement
- Technologies bien connues (HTML, CSS, JS)

Inconvénients :

- Connexion Internet obligatoire
- Moins fluide et réactive que le natif
- Accès limité aux spécificités du système
- Pas d'accès aux capteurs

Reposent sur :

- HTML/CSS/JavaScript
- Affichage via une *Webview* intégrée dans une application native (conteneur)

Avantages :

- Coût réduit
- Développement rapide
- Compétences web classiques suffisantes

Inconvénients :

- Moins performant
- Style graphique éloigné du natif
- Accès limité aux spécificités du système
- Accès restreint à la carte graphique

Frameworks principaux :

- Cordova (PhoneGap)
- Ionic + Angular
- Appcelerator Titanium

Un seul langage pour plusieurs plateformes, tout en conservant l'accès aux API natives.

Exemple principal : Xamarin

- Développement en C#
- Très bonnes performances
- Accès à 100% des API natives
- Forte mutualisation du code

Part de code commun selon l'approche :

- **70–85%** : approche classique (interfaces natives)
- **90–98%** : Xamarin Forms (API graphique indépendante)

Autres alternatives :

- Codename One (Java)
- Qt (C++)
- Windev Mobile (WLanguage)
- C++ Builder

Comparaison frameworks cross-platform

Framework	Langage	Performance	Notes
React Native	JS	Bonne	Large communauté, accès natif via bridge
Flutter	Dart	Très bonne	UI déclarative, compilé en natif
Kotlin Multiplatform	Kotlin	Très bonne	Code partagé, API natives directes

- Développées pour une plateforme spécifique.
- Langages : Java/Kotlin (Android), Swift (iOS).
- **Avantages** : performance et accès complet au matériel.
- **Inconvénient** : maintenance séparée du code.

Les applications natives sont les plus performantes et utilisent pleinement le matériel de l'appareil.

Applications hybrides :

- Code unique pour Android & iOS.
- Frameworks : React Native, Flutter, Ionic.

Applications web mobiles :

- Sites web responsives.
- Exécutées via le navigateur.

Android Studio (natif)

- IDE officiel d'Android.
- Nécessite : JDK, SDK Android, AVD (emulateur).
- Langages : Kotlin (recommande), Java.

React Native (hybride)

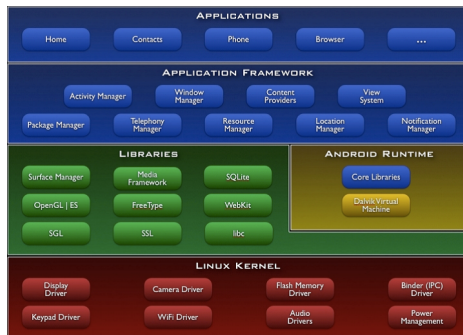
- Créé par Facebook (2015)
- Outils : Node.js, npm/yarn, Expo CLI
- Fonctionne sur Android & iOS

Composant	Rôle
CPU	Exécute les instructions
RAM	Mémoire exécution
Stockage	Contient Android et données
GPU	Accélération graphique
Capteurs	GPS, camera, accéléromètre
Connectivité	Wi-Fi, Bluetooth, 5G

Architecture logicielle d'Android

- 1 Noyau Linux
- 2 HAL (Hardware Abstraction Layer)
- 3 Android Runtime (ART)
- 4 Bibliothèques natives (C/C++)
- 5 Framework Android
- 6 Applications (système et utilisateur)

Architecture logicielle d'Android

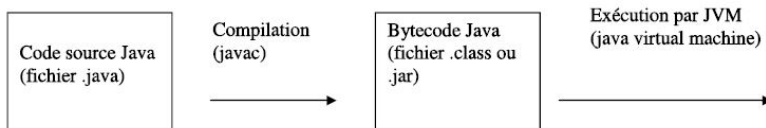


Remarques :

- Les services système (Notification Manager, Location Manager) permettent aux applications d'interagir avec le système.
- ART compile le bytecode en avance (AOT) pour plus de fluidité.

La machine virtuelle Dalvik et Android Runtime (ART)

Rappel : Comment générer un programme exécutable en java standard :



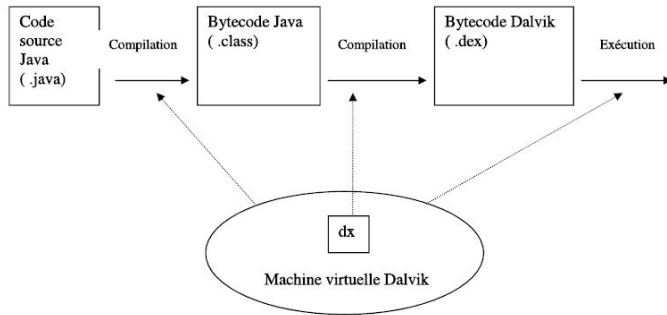
Avec Android :

la version de Java utilisée pour le développement d'applications mobiles est une version réduite, dépourvue de certaines fonctionnalités jugées inutiles sur mobile. Par exemple, la bibliothèque `Swing` n'est pas intégrée.

Avant Android 5 : Dalvik

- Jusqu'à Android 4.x (KitKat), le système utilisait la machine virtuelle **Dalvik**.
- Dalvik est une machine virtuelle spécialement conçue pour les systèmes embarqués.
- Elle est optimisée pour :
 - la gestion de la mémoire,
 - la réduction de la consommation d'énergie,
 - l'exécution de multiples instances (une par application).

La machine virtuelle Dalvik et Android Runtime (ART)



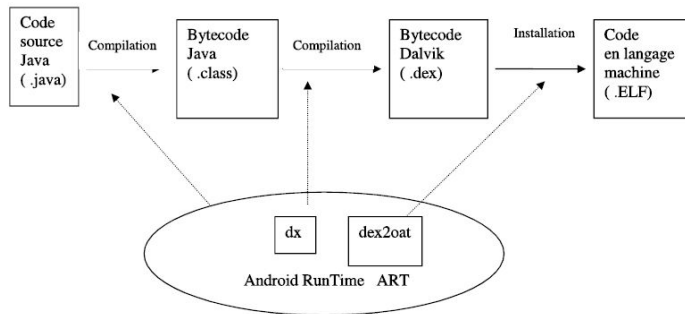
Dx est un programme dont le rôle est de traduire du bytecode Java en bytecode Dalvik. Le code final d'une application est emballé dans un fichier APK (Android Package). Ce code est fabriqué par l'outil AAPT (Android Asset Packaging Tool).

La machine virtuelle Dalvik et Android Runtime (ART)

Depuis Android 5 : ART (Android RunTime) remplace Dalvik à partir d'Android 5 (Lollipop).

- Le bytecode n'est plus traduit en langage machine au moment de l'exécution (just-in-time).
- Il est compilé au moment de l'installation : c'est la compilation **ahead-of-time (AOT)**.
- ART utilise les mêmes fichiers `.dex` que Dalvik.
- L'exécution s'effectue via des fichiers binaires `.ELF` (Executable and Linkable Format).

La machine virtuelle Dalvik et Android Runtime (ART)



- **Avantages :**

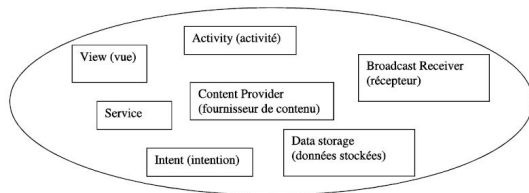
- Les applications sont plus fluides : plus de traduction à la volée (JIT).
- Consommation de batterie réduite grâce à la compilation AOT.

- **Inconvénients :**

- Le code compilé en langage machine occupe davantage d'espace.
- Le temps d'installation est plus long, gênant surtout pour les développeurs.
- ART apporte cependant :
 - un garbage collector amélioré,
 - de meilleures fonctionnalités de mise au point,
 - des messages d'erreurs plus détaillés.

Les composants d'une application Android

Une application Android est un assemblage de composants faiblement couplés.



- Tous ces composants sont regroupés dans un fichier `.apk` (fichier ZIP, Android Package) pour former l'application installable.
- L'application est décrite dans un fichier XML : `AndroidManifest.xml`.
 - On y énumère et configure les composants clés.
 - On y décrit les caractéristiques de l'application.
 - On y spécifie les bibliothèques externes utilisées.
 - On y définit les permissions requises.

Activité (Activity)

- Brique de base d'une interface utilisateur
- Équivalent d'une fenêtre (Windows, Linux) ou d'une boîte de dialogue
- Une activité peut ne pas avoir un interface utilisateur (ex. : certains services)

Fournisseur de contenu (Content Provider)

- Niveau d'abstraction pour toutes les données stockées sur le terminal
- Android encourage le partage de ses propres données avec d'autres applications
- Le Content Provider permet de contrôler la manière dont on accède aux données

Intention (Intent)

- Une intention (*Intent*) est un message système que l'on peut qualifier d'événement.
- Elle peut être émise :
 - par le système pour prévenir les applications d'événements système ;
 - par une application pour annoncer un événement applicatif.

Exemples d'événements système

- Insertion d'une carte SD
- Réception d'un SMS
- Autres notifications système...

Exemples d'événements applicatifs

- *Intent* : « Le logiciel NetSpyR&T démarre »
- *Intent* : « L'utilisateur arrive à Paris » (selon la géolocalisation du terminal)

Service (Service)

- Logiciel autonome prévu pour durer, contrairement aux activités, fournisseurs de contenus ou récepteurs d'intentions.
- Ne nécessite pas d'interface utilisateur.
- Exemples :
 - Service vérifiant périodiquement des mises à jour de flux RSS
 - Service permettant d'écouter une playlist indépendamment de toute activité

Manifeste (Manifest)

- Point de départ de toute application Android.
- Permet de déclarer ce que contient l'application (activités, services, etc.).
- Précise comment ces composants sont reliés au système Android (apparition dans le menu, etc.).
- Indique les permissions requises (accès réseau, caméra, géolocalisation, etc.).

Termes importants : Identifiant uniforme de ressource (URI)

Identifiant uniforme de ressource (Uniform Resource Identifier-URI)

- Courte chaîne de caractères identifiant une ressource sur un réseau réel ou abstrait
- Respecte la norme Internet pour le Web (RFC 3986)

Exemples d'URI

- **URL (Uniform Resource Locator)** : identifie une ressource sur un réseau et fournit les moyens d'agir sur elle ou d'obtenir une représentation.
 - Exemple : `http://www.wikipedia.org/` Identifie la page d'accueil de Wikipédia et permet d'obtenir sa représentation HTML via HTTP.
- **URN (Uniform Resource Name)** : identifie une ressource par son nom dans un espace de noms.
 - Exemple : `urn:isbn:0-395-36341-1` Identifie un livre via son ISBN, sans indiquer où ni comment obtenir une copie réelle.

Conteneur (Container)

- Permet de disposer un ensemble de widgets pour obtenir la présentation désirée
- La plupart des outils de construction d'interfaces graphiques fournissent des gestionnaires de disposition (*layout manager*) organisés en conteneurs

Équipement Android virtuel (Android Virtual Devices - AVD)

- Permet de simuler l'exécution d'un terminal Android sur un ordinateur
- Les terminaux sont personnalisables (version d'Android, type de processeur, espace de stockage, etc.)
- Simplifie le développement et la mise au point des applications

Création d'un projet Android

- Créer un projet à partir d'un exemple fourni par Android Studio
- Créer son propre projet à partir de zéro

Structuration globale des répertoires

- Organisation standard des dossiers dans un projet Android :
 - `app/`
 - `app/manifests/AndroidManifest.xml` : manifeste de l'application
 - `app/java/com.example.app/` : code source Java/Kotlin
 - `app/res/` : ressources (layouts, images, valeurs, etc.)
 - `app/res/drawable/` : images et éléments graphiques (icônes, fonds, etc.)
 - `app/res/layout/` : fichiers XML définissant l'interface utilisateur (layouts)
 - `app/res/values/` : ressources de valeurs (strings, colors, dimens, styles)
 - `app/res/mipmap/` : icônes de l'application à différentes résolutions
 - `app/build.gradle` : fichiers générés par Gradle

Rôle du fichier build.gradle

```
android {  
    compileSdk 33  
    defaultConfig {  
        applicationId "com.example.app"  
        minSdk 21  
        targetSdk 33  
    }  
}
```

Qu'est-ce qu'une activité ?

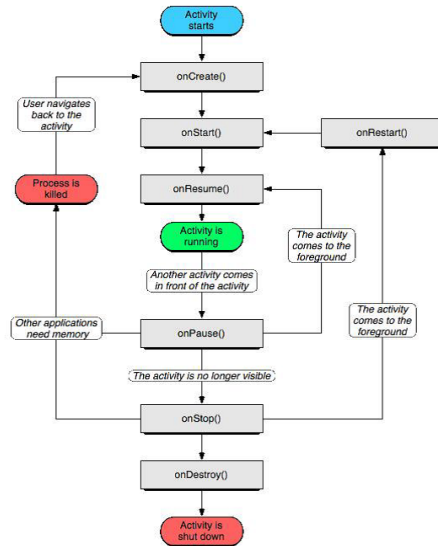
Définition

- Une activité (*Activity*) est une interface utilisateur pour un cas d'utilisation (cf. cours UML)
- Exemple : application d'envoi de SMS
 - Une activité pour choisir un contact dans le répertoire
 - Une activité pour la saisie du message
 - Une activité pour afficher le journal des envois

D'un point de vue opérationnel

- Une activité doit hériter de la classe `AppCompatActivity`
- Une activité est indépendante des autres, **mais** :
 - Il faut désigner une activité de départ (celle utilisée en premier)
 - Il faut chaîner les activités (une activité doit activer la suivante)
- Une activité est liée à un fichier layout XML

Cycle de vie d'une Activity



Cycle de vie d'une activité Android : onCreate()

onCreate()

- Méthode exécutée quand l'activité est créée
- Si c'est la première activité, elle est appelée au lancement de l'application
- Utilisée pour initialiser :
 - la vue XML (`setContentView()`)
 - les fichiers ou données temporaires

Cycle de vie d'une activité Android : `onRestart()` et `onStart()`

`onRestart()`

- Méthode exécutée lorsque l'activité redémarre après un `stop()`
- Appelée lorsque l'application repasse en premier plan après un arrêt prolongé

`onStart()`

- Méthode exécutée après chaque `onCreate()` ou `onRestart()`
- Permet de recharger les données sauvegardées durant le dernier arrêt, si nécessaire

onResume()

- Méthode exécutée après chaque appel de onStart()
- Méthode exécutée à chaque passage en premier plan (si pas de stop())
- Permet, si nécessaire :
 - de gérer la connexion à la base de données
 - de mettre à jour les données modifiées entre temps

onPause()

- Méthode exécutée lorsque :
 - l'utilisateur passe à une autre activité
 - l'utilisateur appelle `finish()` sur cette activité
 - le système a besoin de libérer de la mémoire
- Toujours exécutée avant chaque `onStop()`
- Permet, si nécessaire :
 - sauvegarder les données temporaires
 - gérer la déconnexion à la base de données
- Doit être rapide pour ne pas bloquer la prochaine activité

Cycle de vie d'une activité Android : onStop() et onDestroy()

onStop()

- Méthode exécutée avant chaque mise en sommeil
- Toujours appelée avant onDestroy()
- Permet de libérer certaines ressources

onDestroy()

- Méthode exécutée lors du kill/arrêt de l'activité
- onCreate() devra être appelé à nouveau pour recréer l'activité
- Permet de libérer les fichiers temporaires ou autres ressources

- Ne pas bloquer le thread principal
- Utiliser ViewModel pour conserver l'état sur rotation
- Libérer les ressources dans onStop/onDestroy

Exemple de log du cycle de vie

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    Log.d("CycleVie", "onCreate appele");
}
```

Navigation entre activités

```
Intent intent = new Intent(this, SecondActivity.class);
startActivity(intent);
```

```
// Intent implicite
Intent web = new Intent(Intent.ACTION_VIEW,
    Uri.parse("https://www.google.com"));
startActivity(web);
```

- **LinearLayout** — disposition verticale ou horizontale
- **RelativeLayout** — positionnement relatif
- **ConstraintLayout** — flexible et moderne
- **ScrollView** — défilement vertical **RecyclerView** — listes dynamiques performantes
- **ViewBinding** / **Jetpack Compose** — évite findViewById, UI déclarative

Définition

- XML est un format textuel, structuré et extensible
- **eXtensible Markup Language** : langage de balisage extensible pour structurer des données
- Deux versions principales :
 - 1.0
 - 1.1

Exemple d'interface XML

```
<LinearLayout
  android:orientation="vertical"
  android:gravity="center">
  <TextView
    android:text="Bonjour et bienvenue !" />
  <Button
    android:text="Clique-moi" />
</LinearLayout>
```

Exemple de layout XML dans Android

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3     android:layout_width="fill_parent"
4     android:layout_height="fill_parent"
5     android:orientation="vertical"
6
7     <TextView
8         android:id="@+id/text"
9         android:layout_width="wrap_content"
10        android:layout_height="wrap_content"
11        android:text="Hello, I am a TextView" />
12
13    <Button
14        android:id="@+id/button"
15        android:layout_width="wrap_content"
16        android:layout_height="wrap_content"
17        android:text="Hello, I am a Button" />
18
19    <ImageView
20        android:id="@+id/imageView1"
21        android:layout_width="161dp"
22        android:layout_height="wrap_content"
23        android:layout_weight="0.36"
24        android:src="@android:drawable/sym_action_email" />
25
26 </LinearLayout>
```

Image 1 : exemple de layout

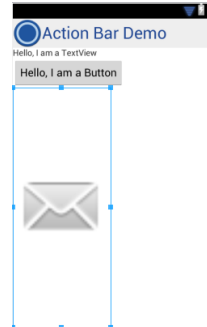


Image 2 : structure XML

Prologue XML (ligne 1)

- `<?xml ... ?>` : entête XML
- `version="1.0"` : version XML 1.0
- `encoding="utf-8"` : format de codage des caractères (ISO/CEI 10646, codage sur 8 bits)

Exemple de layout XML dans Android

11

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3      android:layout_width="fill_parent"
4      android:layout_height="fill_parent"
5      android:orientation="vertical" >
6
7      <TextView
8          android:id="@+id/text"
9          android:layout_width="wrap_content"
10         android:layout_height="wrap_content"
11         android:text="Hello, I am a TextView" />
12
13     <Button
14         android:id="@+id/button"
15         android:layout_width="wrap_content"
16         android:layout_height="wrap_content"
17         android:text="Hello, I am a Button" />
18
19     <ImageView
20         android:id="@+id/imageView1"
21         android:layout_width="161dp"
22         android:layout_height="wrap_content"
23         android:layout_weight="0.36"
24         android:src="@android:drawable/sym_action_email" />
25
26 </LinearLayout>
```

Image 1 : exemple de layout



Image 2 : structure XML

LinearLayout (lignes 2 à 5)

- `<LinearLayout ... >` : spécifie le conteneur (disposition des widgets les uns derrière les autres)
- `xmlns:android="..."` : déclaration de l'espace de noms XML d'Android
- `android:layout_width="..."` : largeur du layout, ici "remplit" le parent (écran)
- `android:layout_height="..."` : hauteur du layout, ici "remplit" le parent (écran)
- `android:orientation="..."` : orientation des widgets, ici verticale

Exemple de layout XML dans Android

11

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3      android:layout_width="fill_parent"
4      android:layout_height="fill_parent"
5      android:orientation="vertical" >
6
7      <TextView
8          android:id="@+id/text"
9          android:layout_width="wrap_content"
10         android:layout_height="wrap_content"
11         android:text="Hello, I am a TextView" />
12
13     <Button
14         android:id="@+id/button"
15         android:layout_width="wrap_content"
16         android:layout_height="wrap_content"
17         android:text="Hello, I am a Button" />
18
19     <ImageView
20         android:id="@+id/imageView1"
21         android:layout_width="161dp"
22         android:layout_height="wrap_content"
23         android:layout_weight="0.36"
24         android:src="@android:drawable/sym_action_email" />
25
26 </LinearLayout>
```

Image 1 : exemple de layout



Image 2 : structure XML

TextView (lignes 7 à 11)

- `<TextView ... >` : le widget défini ici est un label (un texte)
- `android:id="@+id/text"` : identifiant associé à ce widget (cf. code Java)
- `android:layout_width="wrap_content"` : largeur du widget adaptée au texte qu'il contient
- `android:layout_height="wrap_content"` : hauteur du widget adaptée au texte qu'il contient
- `android:text="Hello, I am a TextView"` : texte affiché par le widget

Exemple de layout XML dans Android

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3     android:layout_width="fill_parent"
4     android:layout_height="fill_parent"
5     android:orientation="vertical" >
6
7     <TextView
8         android:id="@+id/text"
9         android:layout_width="wrap_content"
10        android:layout_height="wrap_content"
11        android:text="Hello, I am a TextView" />
12
13    <Button
14        android:id="@+id/button"
15        android:layout_width="wrap_content"
16        android:layout_height="wrap_content"
17        android:text="Hello, I am a Button" />
18
19    <ImageView
20        android:id="@+id/imageView1"
21        android:layout_width="161dp"
22        android:layout_height="wrap_content"
23        android:layout_weight="0.36"
24        android:src="@android:drawable/sym_action_email" />
25
26 </LinearLayout>
```

Image 1 : exemple de layout



Image 2 : structure XML

Button (lignes 13 à 17)

- `<Button ... >` : le widget défini ici est un bouton (un label de texte)
- `android:id="@+id/button"` : identifiant associé à ce widget
- En Java, on va créer une instance de la classe `Button` pour manipuler ce widget :
 - `Button myButton = (Button) findViewById(R.id.button);`

Exemple de layout XML dans Android

11

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3     android:layout_width="fill_parent"
4     android:layout_height="fill_parent"
5     android:orientation="vertical" >
6
7     <TextView
8         android:id="@+id/text"
9         android:layout_width="wrap_content"
10        android:layout_height="wrap_content"
11        android:text="Hello, I am a TextView" />
12
13     <Button
14         android:id="@+id/button"
15         android:layout_width="wrap_content"
16         android:layout_height="wrap_content"
17         android:text="Hello, I am a Button" />
18
19     <ImageView
20         android:id="@+id/imageView1"
21         android:layout_width="161dp"
22         android:layout_height="wrap_content"
23         android:layout_weight="0.36"
24         android:src="@android:drawable/ym_action_email" />
25
26 </LinearLayout>
```

Image 1 : exemple de layout

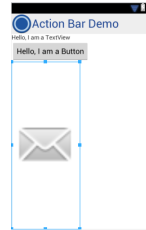


Image 2 : structure XML

ImageView (lignes 19 à 24)

- `<ImageView ... >` : widget ImageView
- `android:id="@+id/imageView1"` : identifiant de l'image
- `android:layout_width="161dp"` : largeur de l'image = 161 dot points
- `android:layout_height="wrap_content"` : prend la hauteur adaptée à l'image
- `android:layout_weight="0.36"` : voir la propriété `android:layout_gravity`
- `android:src="@android:drawable/ym_action_email"` : URI de l'image

```
button.setOnClickListener(v -> {  
    Toast.makeText(this, "Bouton clique !", Toast.LENGTH_SHORT).show  
        ();  
});
```

Avant-propos

- Fichier XML
- Précise l'architecture de l'application
- Chaque application doit en avoir un
- `AndroidManifest.xml` se trouve à la racine du projet

Structure du fichier AndroidManifest.xml

```
<manifest package="com.example.webapp">
  <application>
    <activity android:name=".MainActivity">
      <intent-filter>
        <action android:name="android.intent.action.MAIN"/>
        <category android:name="android.intent.category.LAUNCHER"/>
      </intent-filter>
    </activity>
  </application>
</manifest>
```

- Précise le nom du package Java de l'application (identifiant unique)
- Décrit les composants de l'application :
 - Liste des activités, services, broadcast receivers
 - Précise les classes qui les implémentent
 - Précise leurs capacités (à quels intents ils réagissent)
 - Ceci permet au système de savoir comment lancer chaque partie de l'application afin de respecter le principe de réutilisabilité
- Définit les permissions de l'application :
 - Droit de passer des appels
 - Droit d'accéder à Internet
 - Droit d'accéder au GPS
 - ...
- Précise la version minimale d'Android nécessaire
- Déclare les bibliothèques utilisées
- Déclare des outils d'instrumentation (uniquement pour le développement)

- Seuls deux éléments sont obligatoires :
 - `<manifest>` : contient le package, la version ... Englobe tout le fichier
 - `<application>` : décrit l'application et contiendra la liste de ses composants
- Les données sont passées en tant qu'attribut et non en tant que contenu
- Tous les attributs commencent par `android:` (sauf quelques-uns dans `<manifest>`)

- Au lieu de contenir les données en tant que telles, le fichier `AndroidManifest.xml` peut faire appel à des ressources
- Exemple : `<activity android:icon="@drawable/smallPic" ... >`
- Ces ressources sont définies dans le répertoire `res/` de l'application

- Une application ne peut pas utiliser certaines fonctionnalités sauf si elles sont précisées dans le fichier manifeste
- Il faut donc préciser les permissions nécessaires grâce à : `<uses-permission>`
- Exemples de permissions standards :
 - `android.permission.INTERNET`
 - `android.permission.CAMERA`
 - `android.permission.ACCESS_FINE_LOCATION`
 - `android.permission.CALL_EMERGENCY_NUMBERS`
 - `android.permission.READ_OWNER_DATA`
 - `android.permission.SET_WALLPAPER`
 - `android.permission.DEVICE_POWER`
- Il est également possible de définir ses propres permissions

Permissions courantes

```
<uses-permission android:name="android.permission.INTERNET"/>  
<uses-permission android:name="android.permission.CAMERA"/>  
<uses-permission android:name="android.permission.  
ACCESS_FINE_LOCATION"/>
```

Exemple : WebView avec permission INTERNET

```
WebView webView = findViewById(R.id.webView);  
webView.getSettings().setJavaScriptEnabled(true);  
webView.setWebViewClient(new WebViewClient());  
webView.loadUrl("https://www.google.com");
```

- Les Intent Filter informent le système à quelles *intents* les composants peuvent réagir
- Un composant peut avoir plusieurs filtres
- Exemple : cas d'un éditeur de texte
 - Filtre pour éditer un document existant
 - Filtre pour initier un nouveau document
- Un filtre doit posséder une “action” qui définit à quoi il correspond

Manifeste : Exemple

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.eyrolles.android.multimedia"
    android:versionCode="1"
    android:versionName="1.0">
    <application android:icon="@drawable/icon" android:label="@string/app_name">
        <activity android:name=".CapturePhoto"
            android:label="@string/app_name">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
    <uses-sdk android:minSdkVersion="3" />
    <uses-permission android:name="android.permission.CAMERA"/>
</manifest>
```

Travaux pratiques : WebView et permission INTERNET

Objectif : Activer la permission INTERNET et afficher une page web dans un WebView.

etapes : 1. Créer un projet Java : WebApp. 2. Ajouter la permission INTERNET dans AndroidManifest.xml :

```
<uses-permission android:name="android.permission.INTERNET"/>
```

3. Interface XML (activity_main.xml) :

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/
    android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <WebView
        android:id="@+id/webView"
        android:layout_width="match_parent"
        android:layout_height="match_parent"/>

</LinearLayout>
```

Objectifs pédagogiques :

- Apprendre à utiliser les fichiers multimédias (audio/video) dans Android.
- Manipuler la camera et prendre des photos.
- Gérer les images avec Drawable et Bitmap.
- Comprendre le stockage des ressources dans assets et raw.
- Créer un TP : lecteur audio simple.

Utilisation de MediaPlayer (audio/video)

MediaPlayer permet de lire des fichiers audio ou vidéo.

Exemple : Lecture d'un fichier audio dans res/raw/son.mp3

```
MediaPlayer mediaPlayer = MediaPlayer.create(this, R.raw.son);  
mediaPlayer.start(); // Lecture  
mediaPlayer.pause(); // Pause  
mediaPlayer.stop();  // Arrêt
```

Remarques :

- Les fichiers audio/vidéo doivent être placés dans res/raw/.
- Toujours libérer les ressources :

```
mediaPlayer.release();
```

Caméra et prise de photo

Intent pour capturer une photo :

```
Intent takePictureIntent = new Intent(MediaStore.  
    ACTION_IMAGE_CAPTURE);  
if (takePictureIntent.resolveActivity(getPackageManager()) != null)  
    { startActivityForResult(takePictureIntent, REQUEST_IMAGE_CAPTURE  
        );}
```

Récupérer l'image dans onActivityResult :

```
@Override  
protected void onActivityResult(int requestCode, int resultCode,  
    Intent data) {  
    super.onActivityResult(requestCode, resultCode, data);  
    if(requestCode == REQUEST_IMAGE_CAPTURE && resultCode ==  
        RESULT_OK) {  
        Bundle extras = data.getExtras();  
        Bitmap imageBitmap = (Bitmap) extras.get("data");  
        imageView.setImageBitmap(imageBitmap);}}
```

Gestion des images (Drawable, Bitmap)

Utilisation d'un Drawable :

```
ImageView imageView = findViewById(R.id.imageView);  
imageView.setImageResource(R.drawable.logo);
```

Conversion Bitmap :

```
Bitmap bitmap = BitmapFactory.decodeResource(getResources(), R.  
    drawable.logo);  
imageView.setImageBitmap(bitmap);
```

Remarques :

- Drawable = ressources graphiques dans res/drawable/.
- Bitmap = image manipulable en mémoire (redimension, transformation).

Stockage des ressources (assets, raw)

- **res/raw** : fichiers multimédias accessibles via `R.raw.nom_fichier`
- **assets/** : fichiers arbitraires accessibles via `AssetManager`

Lire un fichier depuis assets :

```
AssetManager am = getAssets();  
InputStream is = am.open("fichier.txt");  
// Lire le contenu
```

TP : Lecteur audio simple

Objectif : Lire un fichier audio et créer un petit lecteur multimedia.

Etapes :

- 1 Créer un projet Java : AudioApp.
- 2 Placer un fichier audio musique.mp3 dans res/raw/.
- 3 Interface XML (activity_main.xml) :

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/
    android"
    android:orientation="vertical"
    android:gravity="center"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <Button
        android:id="@+id/buttonPlay"
        android:text="Play"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>
```

Objectif : Lire un fichier audio et créer un petit lecteur multimédia.

```
<Button  
    android:id="@+id/buttonPause"  
    android:text="Pause"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"/>  
</LinearLayout>
```

① Logique MainActivity.java :

```
MediaPlayer mediaPlayer;  
  
@Override  
protected void onCreate(Bundle savedInstanceState) {  
    super.onCreate(savedInstanceState);  
    setContentView(R.layout.activity_main);  
  
    mediaPlayer = MediaPlayer.create(this, R.raw.musique);  
  
    Button play = findViewById(R.id.buttonPlay);  
    Button pause = findViewById(R.id.buttonPause);  
  
    play.setOnClickListener(v -> mediaPlayer.start());  
    pause.setOnClickListener(v -> mediaPlayer.pause());  
}
```

① Résultat attendu :

- Cliquer sur Play : audio joue
- Cliquer sur Pause : audio arrête

Définition :

Un Intent est un mécanisme de communication utilisé dans Android pour permettre aux composants d'une application ou de différentes applications de communiquer entre eux.

Les Intents permettent notamment de :

- Démarrer une Activity
- Lancer un Service
- Envoyer un Broadcast
- Demander à une autre application d'effectuer une action

Un Intent peut contenir plusieurs informations importantes :

- **Action** : l'opération à effectuer
- **Data** : les données ou la ressource à utiliser
- **Category** : le type de composant capable de traiter l'intent
- **Extras** : informations supplémentaires envoyées
- **Flags** : options qui contrôlent le comportement du lancement

Il existe deux types principaux :

1. Intent Explicite

- L'application précise exactement le composant à appeler.
- Utilisé généralement à l'intérieur de la même application.

2. Intent Implicite

- L'application décrit l'action à réaliser.
- Android choisit automatiquement l'application capable d'effectuer cette action.

Principe

Dans un Intent explicite, on spécifie directement la classe de l'Activity à lancer.

Exemple : ouvrir une deuxième activité

Activity 1

```
Intent intent = new Intent(  
    MainActivity.this,  
    SecondActivity.class  
);  
  
startActivity(intent);
```

Explication

- MainActivity.this : contexte actuel
- SecondActivity.class : activité à ouvrir
- startActivity() : lance l'activité

Principe

Un Intent implicite ne précise pas le composant exact.

Il indique :

- l'action à effectuer
- éventuellement les données à utiliser

Android recherche ensuite les applications capables de répondre à cette demande.

Pour qu'une activité puisse répondre à un **Intent implicite**, elle doit déclarer un **Intent Filter** dans le fichier :

AndroidManifest.xml

Un Intent Filter indique :

- les **actions** supportées
- les **catégories**
- éventuellement les **types de données**

Intent Filter dans AndroidManifest

```
<activity android:name=".SecondActivity">

    <intent-filter>

        <action android:name=
            "com.example.OPEN_SECOND_ACTIVITY" />

        <category android:name=
            "android.intent.category.DEFAULT" />

    </intent-filter>

</activity>
```

Intent implicite vers SecondActivity

Dans **MainActivity** :

```
Intent intent = new Intent(  
    "com.example.OPEN_SECOND_ACTIVITY"  
);  
  
startActivity(intent);
```

Android va rechercher automatiquement l'activité qui possède cet **intent-filter**.

L'action décrit l'opération à effectuer.

Exemples d'actions Android :

- Intent.ACTION_VIEW
- Intent.ACTION_SEND
- Intent.ACTION_DIAL
- Intent.ACTION_MAIN

La donnée indique la ressource sur laquelle l'action doit s'appliquer.

Elle est généralement représentée par une URL.

Exemples :

- tel :
- http ://
- geo :
- mailto :

Exemple Action + Data

```
Intent intent = new Intent(  
    Intent.ACTION_VIEW  
);  
  
intent.setData(  
    Uri.parse("https://www.google.com")  
);  
  
startActivity(intent);
```

Résultat : ouverture du navigateur.

La catégorie fournit des informations supplémentaires sur le type de composant capable de traiter l'intent.

Exemples :

- `Intent.CATEGORY_DEFAULT`
- `Intent.CATEGORY_BROWSABLE`
- `Intent.CATEGORY_LAUNCHER`

Exemple Category

```
Intent intent = new Intent(  
    Intent.ACTION_VIEW  
);  
  
intent.addCategory(  
    Intent.CATEGORY_BROWSABLE  
);  
  
intent.setData(  
    Uri.parse("https://example.com")  
);  
  
startActivity(intent);
```

Les Extras permettent de transmettre des informations supplémentaires entre activités.

Exemples :

- texte
- identifiant utilisateur
- image
- nombre

Exemple Extras

```
Intent intent = new Intent(  
    MainActivity.this,  
    SecondActivity.class  
);  
  
intent.putExtra("username","Ali");  
  
startActivity(intent);
```

Dans l'activité destination :

```
String username =  
getIntent().getStringExtra("username");
```

Les Flags contrôlent le comportement du lancement d'une activité.

Exemples :

- FLAG_ACTIVITY_NEW_TASK
- FLAG_ACTIVITY_CLEAR_TOP
- FLAG_ACTIVITY_SINGLE_TOP

Exemple Flags

```
Intent intent = new Intent(  
    this,  
    MainActivity.class  
);  
  
intent.addFlags(  
    Intent.FLAG_ACTIVITY_NEW_TASK  
);  
  
startActivity(intent);
```

Exemple : Ouvrir le téléphone pour appeler

Une application peut demander à Android d'ouvrir l'application téléphone.

Action utilisée :

Intent.ACTION_DIAL

Code : ouvrir le téléphone

```
Intent intent = new Intent(  
    Intent.ACTION_DIAL  
);  
  
intent.setData(  
    Uri.parse("tel:0600000000")  
);  
  
startActivity(intent);
```

Résultat

- L'application téléphone s'ouvre
- Le numéro est déjà rempli

Exemple : Partage d'une photo

Les applications Android peuvent partager des fichiers (images, vidéos, texte).

Action utilisée :

Intent.ACTION_SEND

Code : partage d'une image

```
Intent shareIntent = new Intent(  
    Intent.ACTION_SEND  
);  
  
shareIntent.setType("image/*");  
  
shareIntent.putExtra(  
    Intent.EXTRA_STREAM,  
    imageUri  
);  
  
startActivity(  
    Intent.createChooser(  
        shareIntent,  
        "Partager l'image via"  
    )  
);
```

- ACTION_SEND : action de partage
- setType("image/*") : type de fichier
- EXTRA_STREAM : fichier à partager
- createChooser : affiche la liste des applications disponibles

Comparaison

Intent Explicite	Intent Implicite
Cible connue	Cible inconnue
Même application	Applications différentes
Ex : ouvrir Activity	Ex : appeler, partager

- Les Intents sont essentiels dans Android.
- Ils permettent la communication entre composants.
- Les Intents explicites servent à naviguer dans l'application.
- Les Intents implicites permettent d'utiliser les fonctionnalités d'autres applications.

Objectifs pédagogiques :

- Comprendre le principe des applications multiplateformes.
- Connaître les avantages et inconvénients des applications hybrides.
- Découvrir les principaux frameworks : React Native, Flutter, Ionic.
- Créer un premier projet hybride avec React Native.

- Un seul code source pour plusieurs plateformes (Android et iOS).
- Utilisation de frameworks qui transforment le code en composants natifs.
- L'interface et la logique sont partagées entre les plateformes.

Exemple : une app React Native fonctionne sur Android et iOS avec le même code.

Avantages :

- Gain de temps : une seule base de code.
- Coût de développement réduit.
- Facile à maintenir et à mettre à jour.

Inconvénients :

- Performance parfois inférieure aux applications natives.
- Accès limité à certaines fonctionnalités natives complexes.
- Taille de l'application plus importante.

Principaux frameworks cross-platform

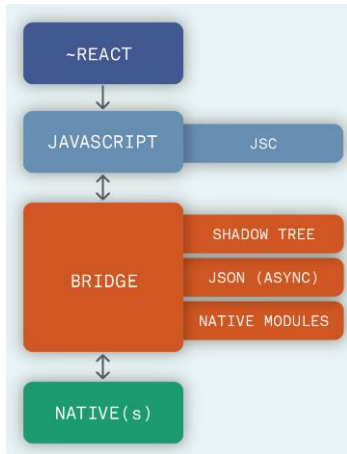
- **React Native** (Facebook) : JavaScript / JSX, communique avec les composants natifs.
- **Flutter** (Google) : Dart, UI compilée en code natif, haute performance.
- **Ionic** : HTML, CSS, JavaScript, s'appuie sur un navigateur web interne.

Choix du framework : selon performance, expérience utilisateur et écosystème.

Qu'est-ce que React Native ?

- Framework open-source de Meta (Facebook 2015) pour créer des applications mobiles multiplateformes (Android, iOS) avec JavaScript et React.
- Permet partage de code, composants natifs et bonnes performances.
- Utilise ses propres composants pour la logique présentation, qui seront mappés en composant OEM Natif de l'appareil mobile.
- Utilise Java Script pour les traitements avec React JS comme framework
- Utile un Bridge Java Script pour accéder aux fonctionnalités Natives

Qu'est-ce que React Native?



- Maîtrise de JavaScript (ES6+)
- Connaissances de base en React (JSX, composants, hooks)
- Node.js et npm/yarn
- Environnement de développement (Android Studio / Xcode ou Expo)

TP : Installation Node.js et Expo CLI

Objectif : Installer l'environnement React Native.

etapes :

- 1 Installer **Node.js** depuis <https://nodejs.org/>.
- 2 Vérifier l'installation :

```
node -v  
npm -v
```

- 3 Installer Expo CLI :

```
npm install --global expo-cli  
expo init MonProjet  
cd MonProjet  
expo start
```

Expo est recommandé pour développement rapide et prototypage.

TP : Création d'un projet React Native "Hello World"

etapes :

- 1 Créer un projet :

```
expo init HelloWorldApp
```

- 2 Choisir le template blank.

- 3 Se rendre dans le dossier du projet :

```
cd HelloWorldApp
```

- 4 Lancer l'application :

```
expo start
```

- 5 Ouvrir sur un simulateur ou smartphone avec l'application Expo Go.

TP : Code "Hello World" React Native

Exemple de code : App.js

```
import React from 'react';
import { StyleSheet, Text, View } from 'react-native';
export default function App() {
  return (
    <View style={styles.container}>
      <Text>Hello World!</Text>
    </View>
  );
}
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```

Résultat attendu : Affichage "Hello World !" centre sur l'écran

Objectifs pédagogiques :

- Comprendre la structure d'un projet React Native.
- Manipuler les composants de base : View, Text, Image, Button.
- Appliquer des styles avec StyleSheet.
- Découvrir les hooks : `useState`, `useEffect`.
- Créer un TP : interface avec compteur et bouton "+1".

Structure minimale d'un projet 1/1

Exemple : structure minimale

```
HelloWorldApp/  
  | - App.js  
  | - package.json  
  | - node_modules/  
  ' - assets/
```

Description des éléments :

- **App.js** :
 - Fichier principal de l'application
 - Contient le composant racine (Root Component)
 - Gère l'affichage initial (UI)
- **package.json** :
 - Fichier de configuration du projet
 - Liste des dépendances (React, React Native, etc.)
 - Contient les scripts (start, build, etc.)

Structure minimale d'un projet

Description des éléments :

- **node_modules/ :**
 - Contient toutes les bibliothèques installées
 - Généré automatiquement avec npm ou yarn
 - Ne doit pas être modifié manuellement
- **assets/ :**
 - Contient les ressources statiques
 - Images, icônes, polices (fonts), etc.

Remarque :

- Cette structure peut évoluer avec la complexité du projet
- On ajoute souvent : `components/`, `screens/`, `services/`

Exemple minimal

React Native utilise JSX pour décrire l'interface utilisateur.

```
import React from 'react';
import { View, Text } from 'react-native';

export default function App(){
  return (
    <View>
    <Text>Hello World</Text>
    </View>
  );
}
```

- View : conteneur
- Text : affichage texte
- Image : image
- TextInput : champ texte
- ScrollView, FlatList : listes et défilement
- TouchableOpacity, Pressable, Button

Composants de base

View : conteneur pour d'autres composants **Text** : afficher du texte **Button** : bouton cliquable **Image** : afficher une image

Exemple :

```
import React from 'react';
import { View, Text, Button, Image } from 'react-native';
export default function App() {
  return (
    <View>
      <Text>Bonjour React Native!</Text>
      <Button title="Cliquer ici" onPress={() => alert('Bouton!')} />
      <Image source={require('./assets/logo.png')} />
    </View>
  );
}
```

Styles avec StyleSheet

React Native utilise un système de styles inspiré de CSS mais sous forme d'objet JS.

Créer des styles :

```
import { StyleSheet } from 'react-native';

const styles = StyleSheet.create({
  container: { flex: 1, padding: 16 },
  row: { flexDirection: 'row', alignItems: 'center' }
});
```

React Native utilise Flexbox pour la mise en page : `flexDirection`, `justifyContent`, `alignItems`.

Utiliser les styles dans les composants :

```
<View style={styles.container}>  
  <Text style={styles.title}>Bonjour React Native!</Text>  
</View>
```

Hooks : useState et useEffect

useState : gérer l'état local

```
import React, { useState } from 'react';  
const [count, setCount] = useState(0);  
  
<Button title="+1" onPress={() => setCount(count + 1)} />  
<Text>Compteur : {count}</Text>
```

useEffect : effets secondaires

```
import React, { useEffect } from 'react';  
  
useEffect(() => {  
  console.log('Composant monte ou mis a jour');  
}, [count]); // se declenche quand count change
```

Objectif : Créer un compteur simple avec bouton "+1"

App.js complet :

```
import React, { useState } from 'react';
import { StyleSheet, Text, View, Button } from 'react-native';
export default function App() {
  const [count, setCount] = useState(0);

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Compteur : {count}</Text>
      <Button title="+1" onPress={() => setCount(count + 1)} />
    </View>
  );
}
```

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    alignItems: 'center',
    justifyContent: 'center',
  },
  title: {
    fontSize: 24,
    marginBottom: 16,
  },
});
```

Resultat attendu : Le compteur s'incrémente à chaque clic sur "+1".

TextInput : pour saisir du texte

```
<TextInput
  placeholder="Nom d'utilisateur"
  value={username}
  onChangeText={setUsername}
/>
```

Switch : pour activer/désactiver une option

```
<Switch
  value={isEnabled}
  onValueChange={setIsEnabled}
/>
```

Picker : sélection parmi plusieurs options

```
<Picker
  selectedValue={selectedValue}
  onChange={itemValue => setSelectedValue(itemValue)}>
  <Picker.Item label="Option 1" value="1" />
  <Picker.Item label="Option 2" value="2" />
</Picker>
```

Composants en React Native

- Un composant = bloc réutilisable
- Permet de structurer l'application
- Deux types :
 - Composants fonctionnels (modernes)
 - Composants classes (anciens)

Exemple

```
function Hello() {  
  return Hello World;  
}
```

- Simple à écrire
- Plus utilisé aujourd'hui
- Compatible avec les Hooks

- Les props (propriétés) permettent de passer des données entre composants React
- Elles sont transmises du composant parent vers le composant enfant
- Les props sont en lecture seule (immutables) : elles ne peuvent pas être modifiées par le composant enfant
- Elles permettent de rendre les composants réutilisables et personnalisables

Exemple

```
function Welcome(props) {  
  return <h1>Hello {props.name}</h1>;  
}
```

- Ici, "props.name" représente une donnée reçue du parent

Exemple

```
// Composant parent
function App() {
  return <Welcome name="Mohammed" />;
}
```

- Le composant parent envoie la prop "name" au composant enfant Welcome
- Le composant enfant utilise cette donnée pour afficher un contenu dynamique
- Communication à sens unique : parent → enfant
- Si la valeur de la prop change, le composant se met à jour automatiquement

- Le **state** représente les données internes d'un composant
- Contrairement aux props, le state peut être **modifié** par le composant lui-même
- Il est utilisé pour gérer des informations qui évoluent dans le temps (compteur, formulaire, etc.)
- Chaque modification du state déclenche automatiquement un **re-render** (mise à jour de l'affichage)
- Exemple : nombre de clics, contenu d'un champ, statut (connecté/non connecté)

Syntaxe

```
const [count, setCount] = useState(0);
```

- **useState** est un Hook qui permet d'ajouter un state dans un composant fonctionnel
- **count** : valeur actuelle du state
- **setCount** : fonction pour mettre à jour cette valeur
- **0** : valeur initiale du state
- Important : on ne modifie jamais directement le state, on utilise toujours la fonction (setCount)

Exemple useState

Code

```
const [count, setCount] = useState(0);

<Button
  title="Click"
  onPress={() => setCount(count + 1)}
/>
```

- Chaque clic sur le bouton appelle **setCount**
- La valeur de **count** augmente de 1
- Le composant se met automatiquement à jour (re-render)
- Résultat : on peut créer facilement un compteur interactif

- La navigation permet de passer d'un écran à un autre dans une application mobile
- Elle est essentielle pour organiser l'application et améliorer l'expérience utilisateur
- Chaque écran représente une vue (page) différente

Types de navigation

- **Stack Navigation** : navigation séquentielle (page par page)
- **Tab Navigation** : navigation par onglets (menu en bas ou en haut)
- Ces deux types sont souvent combinés dans une même application

- Fonctionne comme une pile (stack) : dernier écran ajouté = premier affiché
- L'utilisateur avance ou revient en arrière entre les écrans
- Un bouton "retour" est automatiquement ajouté

Exemple

Accueil → Détails → Paiement

- Très utilisé pour les parcours utilisateurs (ex : consultation d'un produit)

- Permet de naviguer entre les sections principales via des onglets
- Les onglets sont souvent affichés en bas de l'écran (Bottom Tabs)
- Accès rapide sans passer par plusieurs écrans

Exemple

Accueil | Profil | Paramètres

- Idéal pour les fonctionnalités principales d'une application

Commandes

```
npm install @react-navigation/native  
npm install @react-navigation/native-stack  
npm install @react-navigation/bottom-tabs
```

- Ces bibliothèques permettent d'ajouter la navigation dans une application React Native
- D'autres dépendances peuvent être nécessaires selon le projet (gesture-handler, etc.)

Code

```
import { NavigationContainer } from '@react-navigation/native';  
import { createNativeStackNavigator } from '@react-navigation/native-stack';  
  
const Stack = createNativeStackNavigator();
```

- **NavigationContainer** : conteneur principal qui gère toute la navigation
- **Stack** : permet de gérer une pile d'écrans

Code

```
<NavigationContainer>
  <Stack.Navigator>
    <Stack.Screen name="Home" component={HomeScreen} />
    <Stack.Screen name="Details" component={DetailsScreen} />
  </Stack.Navigator>
</NavigationContainer>
```

- Chaque **Stack.Screen** représente un écran de l'application
- **name** : nom utilisé pour naviguer
- **component** : composant React associé à l'écran

Code

```
navigation.navigate('Details');
```

- Permet de passer d'un écran à un autre
- **navigation** est fourni automatiquement par React Navigation
- Le nom doit correspondre à celui défini dans Stack.Screen

Envoi

```
navigation.navigate('Details', { id: 1 });
```

Réception

```
const { id } = route.params;
```

- Permet de transmettre des données entre écrans
- Exemple : envoyer l'ID d'un produit vers une page de détails
- **route.params** contient toutes les données reçues

- Créer 2 écrans :
 - Accueil
 - Détails
- Ajouter un bouton de navigation
- Passer une donnée (nom ou id)
- Afficher cette donnée dans l'écran Détails

- App.js
- screens/
 - HomeScreen.js
 - DetailsScreen.js

Code

```
import { createBottomTabNavigator } from
  '@react-navigation/bottom-tabs';

const Tab = createBottomTabNavigator();
```

- **Tab** : permet de gérer plusieurs écrans accessibles via des onglets
- Chaque onglet correspond à une section principale

Code

```
<NavigationContainer>
  <Tab.Navigator>
    <Tab.Screen name="Home" component={HomeScreen} />
    <Tab.Screen name="Profile" component={ProfileScreen} />
  </Tab.Navigator>
</NavigationContainer>
```

- Chaque **Tab.Screen** représente un onglet
- L'utilisateur peut naviguer librement entre les onglets
- Pas d'ordre imposé comme avec Stack

Code

```
navigation.navigate('Profile');
```

- Permet de changer d'onglet programmatiquement
- Le nom doit correspondre à celui défini dans Tab.Screen
- Utile pour rediriger l'utilisateur

Code

```
<Tab.Screen  
  name="Home"  
  component={HomeScreen}  
  options={{ title: 'Accueil' }}  
>
```

- Possibilité de modifier le titre, les icônes, les styles
- Améliore l'expérience utilisateur

- Créer 2 onglets :
 - Accueil
 - Profil
- Ajouter un contenu différent pour chaque onglet
- Tester la navigation entre les onglets

- App.js
- screens/
 - HomeScreen.js
 - ProfileScreen.js

- Composants = base de React Native
- Props = communication
- State = données dynamiques
- useState = gestion du state
- Navigation = cœur des apps mobiles

Gestion d'état avec useState

useState : gérer l'état local d'un formulaire

```
import React, { useState } from 'react';  
const [username, setUsername] = useState('');  
const [password, setPassword] = useState('');  
  
<TextInput  
  placeholder="Nom d'utilisateur"  
  value={username}  
  onChangeText={setUsername}  
/>  
  
<TextInput  
  placeholder="Mot de passe"  
  value={password}  
  onChangeText={setPassword}  
  secureTextEntry={true}  
/>
```

Stockage local avec AsyncStorage

Exemple : sauvegarder et récupérer le login

```
import AsyncStorage from '@react-native-async-storage/async-storage';

// Sauvegarder
await AsyncStorage.setItem('username', username);

// Recuperer
const savedUser = await AsyncStorage.getItem('username');
```

Remarque : AsyncStorage est asynchrone, utiliser `await` ou `.then()`.

Requetes API avec fetch

```
// Exemple : recuperer des donnees depuis une API
fetch('https://api.example.com/users')
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error(error));
```

Remarque : Toujours gérer les erreurs et les appels asynchrones.

- Avec Expo : `expo-camera`, `expo-location`, `expo-sensors`
- Sans Expo : modules natifs via `react-native-community` ou création de modules natifs (Java/Kotlin, Obj-C/Swift)
- Exemple GPS : `import * as Location from 'expo-location'` ;

TP : Formulaire de connexion

Objectif : Créer un formulaire avec stockage local et message de bienvenue.

App.js complet :

```
import React, { useState, useEffect } from 'react';
import { View, Text, TextInput, Button } from 'react-native';
import AsyncStorage from '@react-native-async-storage/async-storage';

export default function App() {
  const [username, setUsername] = useState('');
  const [savedUser, setSavedUser] = useState('');

  useEffect(() => {
    AsyncStorage.getItem('username').then(user => {
      if(user) setSavedUser(user);
    });
  }, []);
```

TP : Formulaire de connexion

```
const handleLogin = async () => {
  await AsyncStorage.setItem('username', username);
  setSavedUser(username); };
return (
  <View style={{ flex:1, alignItems:'center', justifyContent:'
    center' }}>
    {savedUser ? (
      <Text>Bienvenue, {savedUser}!</Text>
    ) : (
      <>
        <TextInput
          placeholder="Nom d'utilisateur"
          value={username}
          onChangeText={setUsername}
          style={{width:200, borderWidth:1, marginBottom:8}}
        />
      </>
    )
  </View>
);
```

TP : Formulaire de connexion

```
        <Button title="Se connecter" onPress={handleLogin} />
      </>
    )}
  </View>
);
}
```

Résultat attendu : - Saisie du login → stockage local - Affichage "Bienvenue, [username] !" après connexion

Objectifs du chapitre :

- Comprendre ce qu'est SQLite
- Comprendre le rôle de React Native
- Voir comment utiliser SQLite dans une application mobile
- Étudier un exemple simple

Qu'est-ce que SQLite ?

- Base de données relationnelle légère
- Stockée dans un simple fichier
- Ne nécessite pas de serveur
- Très utilisée sur mobile

Exemples d'utilisation :

- Stockage local
- Mode hors ligne
- Petites applications mobiles

Pourquoi utiliser SQLite avec React Native ?

- Sauvegarde locale des données
- Fonctionne sans connexion Internet
- Plus fiable que le stockage simple (AsyncStorage)

Cas typiques :

- Liste de tâches
- Notes
- Applications éducatives

Principe de fonctionnement

- ❶ L'application React Native s'exécute sur le téléphone
- ❷ SQLite stocke les données localement
- ❸ React Native envoie des requêtes SQL
- ❹ SQLite retourne les résultats

Exemple React Native

```
CREATE TABLE tasks (  
  id INTEGER PRIMARY KEY AUTOINCREMENT ,  
  title TEXT  
);
```

Explication :

- Création d'une table tasks
- id : identifiant unique (auto-incrémenté)
- title : texte de la tâche

Ouverture de la base et création

```
import * as SQLite from 'expo-sqlite';

const db = await SQLite.openDatabaseAsync('tasks.db');

await db.execAsync('
  CREATE TABLE IF NOT EXISTS tasks (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT
  );
');
```

Explication :

- `openDatabaseAsync` ouvre (ou crée) la base de données
- `execAsync` exécute du SQL (création de table ici)
- `IF NOT EXISTS` évite les erreurs si la table existe déjà

Insertion d'une tâche

```
await db.runAsync(  
  'INSERT INTO tasks (title) VALUES (?);',  
  ['Apprendre SQLite']  
);
```

Explication :

- runAsync permet d'exécuter INSERT, UPDATE ou DELETE
- Le ? est un paramètre (sécurité contre les injections SQL)
- On insère une nouvelle tâche dans la table

```
const tasks = await db.getAllAsync(  
  'SELECT * FROM tasks;'  
);  
  
console.log(tasks);
```

Explication :

- getAllAsync récupère toutes les lignes
- Résultat : tableau d'objets JavaScript
- Chaque objet représente une tâche

- Les tâches sont enregistrées localement
- Les données persistent après fermeture de l'application
- L'application fonctionne hors ligne

Avantages :

- Rapide
- Simple
- Hors ligne

Limites :

- Pas adapté aux très grosses bases
- Synchronisation serveur à gérer séparément

Conclusion

- SQLite est idéal pour le stockage local
- React Native facilite le développement mobile
- Ensemble, ils permettent de créer des apps mobiles efficaces